The Neuro-symbolic Legal System

A Two-Layer Architecture for Executable Law with Agentic Orchestration

RMS Engineering

Rule Mapping Solutions — Technical Report V2, June 2026

Abstract. We present a two-layer neuro-symbolic architecture for legal knowledge work that combines executable rule trees with an agent-orchestrated knowledge graph. At the lower layer, every legal norm is represented as a *Rulemap*: a symbolic decision tree whose leaves delegate bounded semantic judgements to a language model and whose inner nodes aggregate the leaves under classical Boolean operators with audit-grade traceability. At the upper layer, a *Rule Graph* connects Rulemaps not through semantic similarity, as in conventional retrieval, but through explicit regulatory relations — presupposition, sequence, override — that mirror the procedural structure of the law itself. Above the two layers, a *Conductor* agent operates the graph: it reads from it via constrained tool calls, writes to it through autonomous authoring and enrichment pipelines, and chains atomic assessments into multi-norm verdicts. The system thus turns law from text to be retrieved into process to be executed, and its knowledge base grows as cases are processed. We give a formal account of the schema, the evaluation function, the Conductor's transition relation, and the self-improvement step, and we position the architecture against four neighbouring literatures: Graph-RAG, neuro-symbolic reasoning, agentic RAG, and meta-prompting.

1. Introduction

Two opposite approaches to automating legal reasoning have, until now, both failed in different ways. Large language models, applied to the law as if to any other text, hallucinate citations at rates of 58 to 88 per cent on legal benchmarks [1, 2], lack persistent state across cases, and cannot deliver the deterministic, traceable verdicts that judicial use requires. Classical expert systems, on the other hand, captured procedural logic precisely but were defeated by the knowledge-acquisition bottleneck: each new statute required manual reprogramming, and the resulting rule bases were brittle in the face of legislative change [3].

The structural problem behind both failures is the same. Law is not, in the first instance, a corpus of text to be searched; it is a process to be executed. A single norm is a function from a case to a verdict; a legal domain is a graph of such functions with explicit

dependencies between them — one provision presupposes the satisfaction of another, an exception overrides a rule, a procedural step must precede a substantive evaluation. Neither pure token prediction nor a hand-curated rule base can carry this structure: the first lacks the symbolic substrate, the second lacks the inductive capacity to keep up with a changing legal landscape.

This paper describes an architecture that resolves the tension by stacking two layers. The lower layer is symbolic and executable: each norm is a Boolean rule tree in which language models are used only at the leaves, under bounded prompts, and the aggregation is performed by classical propositional logic with full audit trail. The upper layer is a graph whose edges encode regulatory rather than semantic relationships: not *which norms read alike* but *which norms presuppose, sequence, or override which other norms*. Above both layers an LLM-driven *Conductor* agent operates the graph: it traverses to retrieve, decomposes complex queries into atomic assessments, invokes specialised agents to author or enrich missing nodes, and writes the results back into the graph. The system improves with use.

The contributions are (i) a formal schema for a two-layer architecture that integrates an executable rule layer with a regulatory graph layer (Sections 2–4); (ii) a formalisation of the Conductor's tool-constrained transition relation over the graph (Section 5); (iii) a self-improvement mechanism that lets a single case produce a structural delta to the knowledge base (Section 6); and (iv) a positioning of the architecture against the four nearest research literatures, with a novelty claim that rests on their *integration* rather than on any one of them in isolation (Section 8).

2. The Two-Layer Architecture

Figure 1 sketches the architecture. Two layers carry the knowledge — a layer of Rulemaps and a regulatory graph over them — and a Conductor operates on top, with specialised agents at the side and background pipelines below.

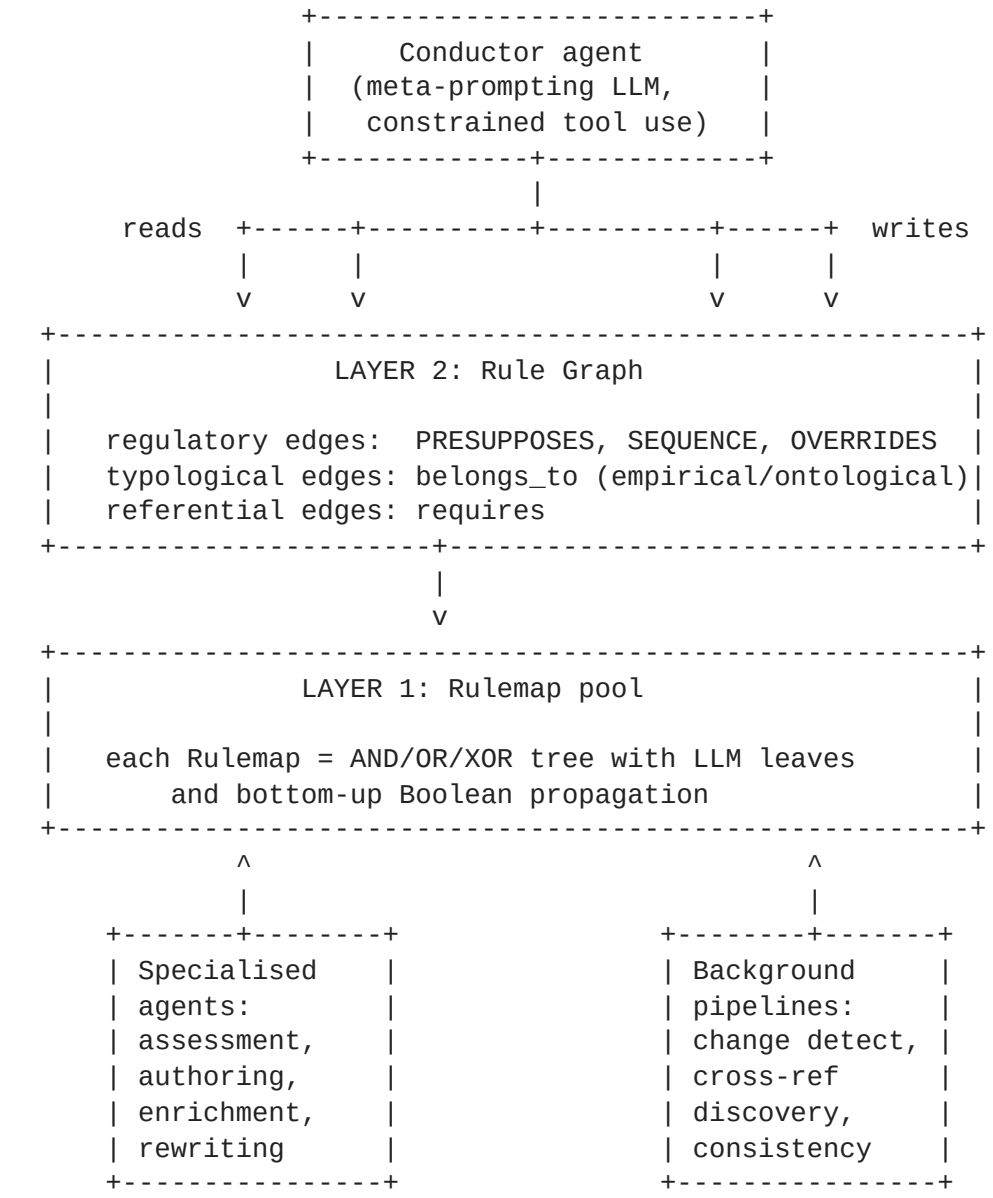

Figure 1. The two-layer architecture. Layer 1 holds executable Rulemaps; Layer 2 the regulatory graph over them. The Conductor reads and writes the graph through constrained tool calls; specialised agents author and enrich Rulemaps; background pipelines keep the graph consistent with its statutory sources.

Two design commitments drive the rest of the paper. First, the lower layer's primitives are *executable*: a Rulemap is not a description of a norm but a procedure for evaluating it. Second, the upper layer's edges are *regulatory*: they encode the relations the law itself draws between its provisions, not the relations a similarity model finds between their texts. Together these two commitments make the system capable of deterministic verdict production while remaining navigable by an agent that does not know the corpus by heart.

3. Layer 1 — The Rulemap as Assessment Agent

We model a single norm as a Rulemap: a labelled rooted tree whose nodes carry logic and whose leaves carry semantic predicates. Formally, we treat the entire corpus as a labelled

multigraph

$$G = (V, E, \lambda, \sigma)$$

with node set V partitioned by function:

$$V = V_p \cup V_s \cup V_c \cup V_\ell$$

where V_p are *examination nodes*, V_s structure nodes (statutory hierarchy), V_c cluster nodes, and V_ℓ legal sources. A single Rulemap is the subtree of V_p rooted at a given norm.

An examination node $v \in V_p$ is a 7-tuple

$$v = \langle \text{id}, \tau, \ell, q, \mathbf{k}, s, \mathbf{e} \rangle$$

consisting of identifier, human-readable title τ, logic type $\ell \in \{\wedge, \vee, \oplus\}$, natural-language query definition q (the prompt against which a leaf is subsumed), keyword list $\mathbf{k}$, summary s, and embedding $\mathbf{e} \in \mathbb{R}^d$ under a fixed encoder.

3.1 Evaluation

Let $\mathbb{T} = \{\top, \bot, ?\}$ be the verdict lattice, and let $\mathscr{F}$ denote the case facts. The evaluation function is defined recursively:

$$\text{eval}(v, \mathscr{F}) = \text{LLM}(q(v), \mathscr{F}) \quad \text{if children}(v) = \varnothing,$$

$$\text{eval}(v, \mathscr{F}) = \begin{cases} \bigwedge_{c \in \text{children}(v)} \text{eval}(c, \mathscr{F}) & \text{if } \ell(v) = \wedge, \\ \bigvee_{c \in \text{children}(v)} \text{eval}(c, \mathscr{F}) & \text{if } \ell(v) = \vee, \\ \bigoplus_{c \in \text{children}(v)} \text{eval}(c, \mathscr{F}) & \text{if } \ell(v) = \oplus. \end{cases}$$

Two short-circuit rules apply upward whenever a leaf produces a verdict. Let $c \in \text{children}(v)$:

$$\ell(v) = \wedge \ \wedge \ \text{eval}(c, \mathscr{F}) = \bot \ \Rightarrow \ \text{eval}(v, \mathscr{F}) = \bot,$$

$$\ell(v) = \vee \ \wedge \ \text{eval}(c, \mathscr{F}) = \top \ \Rightarrow \ \text{eval}(v, \mathscr{F}) = \top.$$

A Rulemap is therefore not merely a piece of declarative data; it is, in operational terms, an *agent*. Given a case, it traverses its own tree, calls the LLM as a subsumption oracle at the leaves, composes the verdicts upward under its declared logic types, and returns a verdict together with a full audit trail of which leaves were evaluated, which produced

which result, and which children were skipped under a short-circuit. The agent abstraction is important for Layer 2: the Rule Graph is a graph of agents, not a graph of documents.

4. Layer 2 — The Regulatory Rule Graph

The defining choice of the upper layer is that its edges encode *regulatory* relationships, not semantic similarity. A semantic graph would record that § 7 SGB II and § 22 SGB VIII read alike; the Rule Graph records, instead, that the former *presupposes* certain conditions, that the latter *sequences* after them in a procedural sense, or that a special rule *overrides* a general one. The graph thus mirrors the structure that the law itself draws between its provisions.

Formally, we extend the edge multiset E with three regulatory relations beyond the structural and referential edges that we inherited from V1:

$$\text{PRESUPPOSES, SEQUENCE, OVERRIDES} : V_p \times V_p \to \mathscr{P},$$

each a typed partial function carrying its own property record. PRESUPPOSES marks a directional dependency: the evaluation of one Rulemap requires a positive verdict from another. SEQUENCE records procedural order: one assessment must occur before another. OVERRIDES captures the *lex specialis* and *lex posterior* relations through which specific or later norms displace general or earlier ones. Property records carry the modality and confidence of each relation and a pointer to the textual or doctrinal source that licences it.

This regulatory layer does not replace the typological and thematic structure of V1. The two-knowledge model — an empirical clustering grown from text similarity, an ontological tree of legal categories — remains in place as the typological axis. The new regulatory edges add a *procedural* axis that the cluster spaces do not capture. Retrieval, traversal, and reasoning can use any axis or any combination: a Conductor faced with a multi-step query can navigate by topic in one moment and by presupposition in the next.

The asymmetry between semantic and regulatory edges matters in practice. A semantic similarity score does not tell an agent whether to *execute* Rulemap A before or after Rulemap B; a SEQUENCE edge does. A semantic similarity score cannot say that one provision rules out the application of another; an OVERRIDES edge can. By making these relations explicit, the Rule Graph lifts decisions out of the LLM's prompt context and into a substrate where they can be verified, audited, and edited independently of any single inference.

5. The Conductor

The third architectural component is the Conductor: an LLM-driven agent that operates the Rule Graph through a bounded vocabulary of tool calls. Where Layer 1 evaluates a single norm and Layer 2 records relations between norms, the Conductor decides which norms to retrieve, in which order to execute them, and what to do if the graph does not yet contain the Rulemap a case requires.

We model the Conductor as a discrete-time controller. Let π_t denote the protocol state at step t — the set of Rulemaps currently active, the evaluated nodes within them, the open facts in $\mathscr{F}_t$, and the satisfied or open regulatory dependencies. Let retrieve(π_t, G) denote a graph query that returns the sub-graph relevant to the current state. The Conductor evolves by

$$\pi_{t+1} = \delta(\pi_t, \mathscr{F}_t, \text{retrieve}(\pi_t, G)),$$

where δ is an LLM-driven decision step constrained to a fixed tool grammar. The Conductor never emits free-form action: it must select one of a small set of typed operations — `query_graph`, `invoke_assessment` on a Rulemap, `request_authoring` for a missing one, `request_enrichment`, `ask_user`, `respond`. Validity of the chosen operation against the graph schema is checked symbolically before execution; the LLM proposes, the symbolic substrate disposes. This is the same architectural pattern that drives the leaf-level evaluation in Layer 1 — bounded inference, symbolic verification — lifted to the orchestration scale.

The Conductor's relation to meta-prompting [4] is instructive. Classical meta-prompting uses an LLM to write the prompts for further LLM calls and accumulates knowledge through chain-of-thought generated on the fly. The Conductor inherits the orchestration pattern but breaks with the generation-on-the-fly assumption: the knowledge it composes is *retrieved from a persistent graph*, not generated each time. This grounds the orchestration in verified, auditable structure and gives the system the persistence property that pure LLM agents notoriously lack. Across cases, the Conductor's effective memory is the graph itself.

6. Self-Improving Graph

A retrieval system that only reads is bounded by the corpus it was given. The Conductor, by contrast, can also write. When the graph does not contain the Rulemap a case requires, the Conductor invokes the authoring agent on the relevant statutory source; when an existing Rulemap is under-specified for the case at hand, it invokes the enrichment agent to attach

additional context; when it discovers a new regulatory edge during traversal, it adds it. Each case is therefore a potential source of structural growth.

We formalise the per-case update as a delta over the graph. Let s_t be the statutory source under consideration at step t and n_t the Rulemap currently under evaluation. The Conductor's authoring, enrichment, and connection actions produce

$$\Delta_t = \text{author}(s_t) \cup \text{enrich}(n_t) \cup \text{connect}(G_t),$$

and the graph evolves as

$$G_{t+1} = G_t \cup \Delta_t,$$

subject to a validation step. The validation is twofold: a symbolic check that Δ_t respects the graph schema (no edge type used outside its declared domain, no Rulemap inserted without a logic-typed root), and an optional human review for novel patterns flagged by the Conductor. The default policy admits low-risk additions automatically and queues higher-risk ones for review, with the threshold a configurable property of the deployment.

Beyond the case-driven delta, the graph is also maintained by background pipelines that operate without an active user query. These pipelines detect changes in legal sources and propagate them to affected sub-graphs; discover implicit cross-references by combining rule-based extraction of explicit citations with an LLM-assisted detection step for implicit dependencies; and run periodic consistency checks across the regulatory edges. Each pipeline produces deltas of the same form as Δ_t above and feeds them into the same validation step.

The architectural commitment is that the graph never stands still. It is, in a strict sense, a living artefact: it accumulates from cases, it tracks the source statutes, it discovers structure that was previously implicit. Each processed case makes the system richer for the next one.

7. Three-Stage Retrieval, Revisited

A useful instance of the Conductor's behaviour is the retrieval protocol that emerges naturally from the two-layer architecture. Given a user query, retrieval proceeds in three semantic stages, each consuming the output of the previous one and operating at a higher level of specificity.

In the first stage, *empirical*, the Conductor localises the case in the corpus. It uses a combination of keyword scoring over V_p, a cluster scorer over the empirical sub-graph of V_c, a vector search over both, and a scorer over the ontological sub-graph. The four signals

are fused into a relevance score and the top results are auto-expanded into a compact context fragment of structured form — the rule trees rooted at the top candidates rendered with logic symbols, the matching clusters expanded into summary and exemplars. This stage answers *where* in the corpus the case sits.

In the second stage, *ontological*, the Conductor consults the ontological classification of the candidate norms to retrieve the applicable *examination schema* — the order in which the children of the rule-tree root should be evaluated and the precedence of entitlement-creating versus entitlement-excluding elements. The new regulatory edges (PRESUPPOSES, SEQUENCE, OVERRIDES) refine this schema with cross-norm constraints: a presupposition introduced by another Rulemap must be discharged before the current one can produce $\top$, and so on. This stage answers *how* the case should be examined.

In the third stage, *subsumption*, the Conductor invokes the Rulemap agent of the selected norm, populating $\mathscr{F}$ with the case facts. The Rulemap's evaluation proceeds as in Section 3; whenever its leaves are unable to decide for lack of information, the Conductor surfaces a bundled clarification request to the user. Propagation and short-circuiting carry the leaves' verdicts to the rule-tree roots, and the regulatory edges of Layer 2 forward verdicts from one Rulemap to the next where presupposition or sequence requires it. This stage answers *satisfied?*

The protocol is not hard-coded into the Conductor. It is a stable trajectory the Conductor settles into because the graph schema makes it the most natural sequence of tool calls. Different graph structures will admit different protocols; the architecture is general.

8. Related Work and Technical Novelty

The architecture sits at the intersection of four nearby research literatures. *Graph-RAG* retrieves from a knowledge graph rather than from a flat passage corpus, exploiting structural relations between entities to surface multi-hop context that flat retrieval misses [5, 6]. Our Rule Graph is a Graph-RAG substrate, but with the unusual property that its edges encode *regulatory* structure rather than semantic similarity. A retrieval that follows a PRESUPPOSES edge follows a real legal dependency; a retrieval that follows a semantic similarity edge follows a guess.

Neuro-symbolic systems combine learned models with symbolic structure and have been shown to outperform pure LLMs on tasks that require logical consistency or formal verification [7]. Our Layer 1 is neuro-symbolic by design: the LLM is used as a bounded subsumption oracle at the leaves, and propositional logic does all aggregation. The verdict at the root is, by construction, deterministic in the leaves; the trace from leaves to root is the audit trail.

Agentic RAG embeds autonomous decision-making into retrieval pipelines, allowing the retriever to plan, reflect, and self-correct rather than executing a fixed query template [8]. Our Conductor is an agentic retriever in this sense, but it operates over a typed graph rather than a vector store, and its action vocabulary includes graph writes — authoring, enrichment, connection — in addition to graph reads.

Meta-prompting uses an LLM to orchestrate sub-prompts and sub-agents [4]. Classical meta-prompting generates the orchestration knowledge through chain-of-thought; the Conductor inherits the orchestration pattern but reads its knowledge from a persistent graph, which is what allows the system to accumulate across cases rather than starting from scratch each time.

The novelty claim is not located in any one of these literatures but in their *integration*. Graph-RAG is grounded in a graph whose edges encode regulatory structure; neuro-symbolic execution is grounded in rule trees that admit deterministic propagation; agentic RAG operates with read-and-write graph operations; meta-prompting is grounded in a persistent knowledge graph that the Conductor can extend. Each component has antecedents; the combined architecture is, to our knowledge, novel.

9. Properties Beyond the State of the Art

The architecture confers four properties that pure LLM-based legal systems lack. The first is *controllability*. Every verdict decomposes into a finite sequence of Boolean leaf evaluations, propositional aggregation steps, and graph-traversal operations. The decomposition is reconstructable from the audit trail and therefore reviewable by the human users of the system, and it satisfies the kind of transparency requirement that the EU AI Act imposes on automated high-risk decisions [9].

The second is *token efficiency*. Because the Conductor retrieves only the sub-graph relevant to the current state, the context provided to any single LLM call remains small. This avoids the well-documented degradation that affects long-context inference [2] and reduces operating cost. Progressive sub-graph loading is, in a sense, the structural counterpart of the algorithmic gains that retrieval-augmented generation made over fully in-context approaches: relevance is determined by graph structure, not by an attention pattern.

The third is *multi-hop reasoning*. Legal reasoning chains across interdependent norms — one provision presupposes another, a procedural step sequences before a substantive one. In a pure LLM system these dependencies are inferred at inference time, often incorrectly. In the Rule Graph they are encoded in the typed edges, and the Conductor resolves execution order by traversing the graph rather than by inferring it.

The fourth is *self-improvement*. Each case processed has the potential to add Rulemaps, enrich existing ones, or discover new regulatory edges. Over time the graph grows in coverage without corresponding manual effort, subject to the validation step described in Section 6. We note, finally, that the architecture is not built from theoretical premises: it extends a methodology that has been deployed commercially for over twenty years and that today operates several hundred Rulemaps in production use across German civil, administrative, and criminal law. The frontier element is the orchestration and self-improvement layer above this existing base, not the base itself.

10. Discussion and Outlook

Three questions are open in the present design. The first concerns the expressiveness of the regulatory edge schema. We have argued that PRESUPPOSES, SEQUENCE, and OVERRIDES capture the main procedural relations of substantive law, but procedural law itself — with its competence rules, deadlines, and procedural defaults — may require further edge types. The right way to find out is to test the schema against a procedural domain at scale; whether it generalises or breaks is an empirical question.

The second question concerns automated detection of dependencies. Explicit statutory references can be extracted with high precision from the text, but implicit dependencies — a definition in one provision that conditions the interpretation of another, a procedural prerequisite that is not named — require reasoning. We expect that a hybrid pipeline combining rule-based extraction with LLM-assisted detection under expert validation will be the right shape, but the achievable precision-recall trade-off is the highest-uncertainty parameter in the system.

The third question concerns the Conductor's behaviour under partial information. A Conductor that traverses a complete graph is conceptually clean; a Conductor that traverses a graph it knows to be incomplete — and that must therefore decide whether to author the missing Rulemap on the fly, defer to a human, or proceed with reduced confidence — is the realistic case. The constrained tool vocabulary we have described admits these choices, but the policies that choose between them are an open research topic.

Beyond these three questions the architecture suggests two longer-term directions. The first is cross-jurisdictional deployment: the same graph schema, populated from a different legal corpus, should support analogous reasoning over the law of any other European jurisdiction. The second is multilingual operation: with the LLM oracle abstracted behind the leaf interface, the language of the user query and the language of the underlying statutes can in principle decouple. Both directions test the architectural commitments of this paper at a larger scale.

11. Conclusion

We have described a two-layer neuro-symbolic architecture for legal knowledge work. At the lower layer, every norm is an executable Rulemap whose verdict decomposes into Boolean leaves and a propositional aggregation. At the upper layer, a graph connects Rulemaps through typed regulatory relations — presupposition, sequence, override — that mirror the structure of the law itself. Above the layers, an LLM-driven Conductor operates the graph through constrained tool calls, retrieves the sub-graph relevant to a case, decomposes complex queries into atomic assessments, and writes the results of authoring and enrichment back into the graph. The system is deterministic at the leaves, transparent in its verdicts, structured in its multi-hop reasoning, and capable of growing its own knowledge base from the cases it processes.

The decisive frontier in this architecture is not the language model but the system around it. By giving the model bounded tasks at the leaves, a typed substrate to reason over, and an orchestration layer that can both read and write the knowledge base, we obtain a system that turns law from a corpus to be searched into a process to be executed.

References

1. M. Dahl, V. Magesh, M. Suzgun, D. E. Ho. *Hallucinating Law: Legal Mistakes with Large Language Models are Pervasive.* Stanford HAI / preprint, 2024.
2. N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. *Lost in the Middle: How Language Models Use Long Contexts.* Transactions of the Association for Computational Linguistics, 2024.
3. J. Cullen, A. Bryman. *The Knowledge Acquisition Bottleneck: Time for Reassessment?* Expert Systems, 1988.
4. S. Hong, M. Zhuge, J. Chen et al. *MetaGPT: Meta-Prompting for Multi-Agent Collaborative Frameworks.* ICLR, 2024.
5. B. Peng, Y. Zhu, Y. Liu et al. *Graph Retrieval-Augmented Generation: A Survey.* Preprint, 2024.
6. H. Han, T. Wang, X. Han et al. *Retrieval-Augmented Generation with Graphs.* Preprint, 2025.
7. B. Colelough, W. Regli. *Neuro-Symbolic AI in 2024: A Systematic Review.* Preprint, 2025.
8. A. Singh, A. Ehtesham, S. Kumar, T. T. Khoei. *Agentic Retrieval-Augmented Generation: A Survey on Agentic RAG.* Preprint, 2025.
9. European Parliament and Council. *Regulation (EU) 2024/1689 (Artificial Intelligence Act).* Official Journal of the European Union, 2024.

10. D. Merigoux, N. Chataing, J. Protzenko. *Catala: A Programming Language for the Law.* Proc. ACM Program. Lang. 5 (ICFP), 2021.
11. L. T. McCarty. *Reflections on TAXMAN: An Experiment in Artificial Intelligence and Legal Reasoning.* Harvard Law Review, 1977.
12. P. Lewis et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* NeurIPS, 2020.
13. S. Hogan et al. *Knowledge Graphs.* ACM Computing Surveys, 2021.